

Resumen Definitivo de Técnicas Digitales 2: STM32 y ARM Cortex-M3

Para romperla en los parciales ;)

¡Hola! Si estás leyendo esto, es porque se vienen los parciales y querés repasar todo para que te vaya excelente. No te preocupes, esto es más fácil de lo que parece. Vamos a ir paso a paso, con analogías y ejercicios para que todo quede súper claro. ¡Éxitos!

!!! SECCIÓN 1: Arquitectura ARM Cortex-M3 (TEORÍA) !!!

¿Qué es el ARM Cortex-M3?

Imaginate que el ARM Cortex-M3 es el "cerebro" de una computadora diminuta. Es el procesador que está adentro de nuestro querido microcontrolador STM32F103. Su trabajo es ejecutar instrucciones rápidamente, pero no está solo: tiene memoria y periféricos a su disposición.

Registros principales

- **R0 a R12:** Son los bolsillos del procesador. Guardan datos temporales (números, direcciones) para trabajar rápido.
- **SP (Stack Pointer - R13):** Apunta a la "pila" en memoria. Hay dos: **MSP** (Main Stack Pointer, usado por defecto y en interrupciones) y **PSP** (Process Stack Pointer, para tareas del usuario). Analogía: es como un anotador donde guardás por dónde ibas cuando te interrumpen.
- **LR (Link Register - R14):** Cuando llamás a una función, este registro guarda el camino de vuelta.
- **PC (Program Counter - R15):** Es el dedo que señala qué instrucción estás leyendo ahora mismo.
- **xPSR (Program Status Register):** Guarda banderitas (flags) de estado, como si una suma dio cero, si dio negativo, o qué interrupción se está ejecutando.

Modos de operación y Privilegios

- **Thread Mode vs Handler Mode:** El Thread Mode es el modo de trabajo "normal". El Handler Mode es el "modo emergencia", cuando ocurre una excepción o interrupción. ¡Siempre se ejecuta en modo privilegiado!
- **Privileged vs Unprivileged:** Privilegiado es como ser "Administrador" en la compu (podés hacer todo). Unprivileged es ser un usuario común (no podés tocar registros críticos).

Arquitectura Harvard Modificada y Pipeline

- **Harvard Modificada:** Tiene dos caminos (buses) separados: uno para leer instrucciones (código) y otro para leer datos. ¡Así puede hacer ambas cosas a la vez sin chocarse! Es "modificada" porque comparten un mismo mapa de memoria global.

- **Pipeline de 3 etapas:** Mientras **Ejecuta** una instrucción, está **Decodificando** la siguiente, y **Buscando** (Fetch) la que le sigue. Analogía: una línea de ensamblaje en una fábrica.

Mapa de Memoria (Memory Map)

El Cortex-M3 ve todo como una gran tira de direcciones de 4 GB:

- **Flash (0x0800_0000):** Donde vive tu código (ROM).
- **SRAM (0x2000_0000):** Memoria de trabajo (RAM) para variables.
- **Periféricos (0x4000_0000):** Donde están los registros de GPIO, TIM, UART, etc.
- **Periféricos del Core (0xE000_E000):** Cosas internas del cerebro, como el NVIC y SysTick.

Bare Metal vs CMSIS

Bare Metal puro es usar punteros pelados: `*(volatile uint32_t*)(0x40010800) = 1;`. Es duro y propenso a errores. **CMSIS** es la librería oficial que usa structs (estructuras) en C para hacernos la vida más fácil. En lugar de punteros feos, hacemos `GPIOA->ODR = 1;` ¡Mucho mejor!

Ejercicios para Practicar (Sec. 1)

Ejercicio 1: ¿Qué registro se usa para saber a dónde volver después de llamar a una función?

Ejercicio 2: Si tu programa entra a una interrupción, ¿en qué modo y nivel de privilegio se encuentra?

Soluciones: 1) El LR (Link Register - R14). 2) Entra en Handler Mode, y siempre con nivel Privileged (Administrador).

!!! SECCIÓN 2: Sistema de Relojes (RCC) y GPIO !!!

El Árbol de Relojes (Clock Tree)

Para que algo funcione, necesita un "latido" eléctrico.

- **HSI:** Oscilador interno de 8MHz. Viene por defecto.
- **HSE:** Cristal externo, suele ser de 8MHz. Es más preciso.
- **PLL:** Multiplicador mágico. Agarra los 8MHz y los multiplica (por ejemplo, x9) para llegar al máximo del micro: **72MHz**.

Jerarquía de Buses y RCC

El procesador corre a 72MHz en el bus principal (**AHB**). De ahí se ramifica:

- **APB2 (High Speed):** Hasta 72MHz. Acá están GPIO, ADC1, USART1, TIM1.
- **APB1 (Low Speed):** Hasta 36MHz. Acá están TIM2, USART2, I2C.

¡IMPORTANTE! Los periféricos están APAGADOS por defecto para ahorrar energía. Hay que prenderles el reloj usando el registro `RCC->APB2ENR` (para APB2) o `RCC->APB1ENR` (para APB1). Analogía: es como encender la llave de luz de la habitación donde vas a trabajar.

GPIO (Pines de entrada/salida)

Cada puerto (GPIOA, GPIOB, etc.) tiene 16 pines. Para configurarlos usamos CRL (pines 0 al 7) y CRH (pines 8 al 15). ¡Cada pin usa 4 bits para configurarse!

Valor Hexadecimal (4 bits)	Configuración del Pin
0x0	Entrada Analógica (ADC)
0x4	Entrada Flotante (Input Floating, estado por defecto)
0x8	Entrada Pull-up / Pull-down
0x1, 0x2, 0x3	Salida Push-Pull (distintas velocidades, ej: 0x2 = 2MHz)
0xB	Salida Alternate Function (PWM, TX UART)

Registros de Datos

- ODR (Output Data Register): Para escribir los 16 pines de una.
- IDR (Input Data Register): Para leer el estado de los pines.
- BSRR / BRR: Para prender o apagar pines individuales muy rápido sin afectar a los demás.

Ejemplo de Código: Parpadear LED en PC13

```
// 1. Prender el reloj del GPIOC en APB2 (Bit 4)
RCC->APB2ENR |= (1 << 4);

// 2. Configurar PC13 como salida (CRH, porque 13 > 7)
// El pin 13 empieza en el bit 20 (porque (13-8)*4 = 20)
GPIOC->CRH &= ~(0xF << 20); // Limpiar esos 4 bits
GPIOC->CRH |= (0x2 << 20); // Poner 0x2 (Salida Push-Pull 2MHz)

// 3. Prender y apagar el LED (PC13)
GPIOC->ODR ^= (1 << 13); // XOR para cambiar estado (Toggle)
```

Ejercicios para Practicar (Sec. 2)

Ejercicio 1: ¿Cómo configuras el pin PA0 como entrada Pull-Up?

Ejercicio 2: Si un pin está en ODR = 1, y quiero apagarlo usando BSRR, ¿qué bit tengo que escribir?

Soluciones: 1) PA0 usa CRL (bits 0 a 3). Pongo el valor 0x8 ahí. Y para que sea Pull-Up, además debo poner un 1 en el ODR de ese pin (GPIOA->ODR |= 1;). Si pusiera un 0, sería Pull-Down. 2) Para apagar el pin 0 con el BSRR, tengo que poner un 1 en la parte de RESET, es decir en el bit 16. GPIOA->BSRR = (1 << 16);.

!!! SECCIÓN 3: SysTick (Base de Tiempos) !!!

¿Qué es SysTick?

Analogía: Es como un temporizador de cocina integrado adentro del micro. Lo ajustás para que suene cada cierto tiempo, y cuando llega a 0, activa una alarma (interrupción) y vuelve a arrancar.

Los Registros del SysTick

- **CTRL:** Tiene los controles básicos. El bit ENABLE lo prende, TICKINT habilita la interrupción, y CLKSOURCE elige de dónde viene el reloj (1 = reloj del procesador).
- **LOAD:** El valor desde donde empieza a contar hacia abajo.
- **VAL:** El valor actual del contador.

La Fórmula de LOAD

La fórmula para saber qué número poner en LOAD es:

$$\text{LOAD} = (T_{\text{deseado}} \times f_{\text{clock}}) - 1$$

Donde T_{deseado} es el tiempo en segundos, y f_{clock} es la frecuencia del SysTick.

Tiempo deseado	LOAD a 72MHz	LOAD a 8MHz
1 ms (0.001 s)	71.999	7.999
10 ms (0.01 s)	719.999	79.999
1 μ s (0,000001 s)	71	7

Cuando SysTick llega a cero, automáticamente llama a la función SysTick_Handler(void).

Ejemplo: Función delay_ms()

```
volatile uint32_t ticksDelay = 0;

void SysTick_Handler(void) {
    if (ticksDelay > 0) {
        ticksDelay--;
    }
}

void delay_ms(uint32_t ms) {
    // Configura SysTick para 1ms a 72MHz
    SysTick_Config(72000);
    ticksDelay = ms;
    while(ticksDelay > 0); // Esperar a que llegue a cero
}
```

Ejercicios para Practicar (Sec. 3)

Ejercicio 1: Si tu procesador va a 72MHz, ¿qué valor ponés en LOAD para tener una interrupción cada 5 ms?

Ejercicio 2: ¿Qué hace la función de CMSIS SysTick_Config(uint32_t ticks)?

Soluciones: 1) $\text{LOAD} = (0.005 \times 72.000.000) - 1 = 360.000 - 1 = 359.999$. 2) Configura el registro LOAD, pone el VAL en 0, selecciona el reloj del core, habilita la interrupción (TICKINT) y arranca el contador (ENABLE) todo en un solo paso.

⋮ SECCIÓN 4: Interrupciones Externas (EXTI) y NVIC ⋮

¿Qué es una Interrupción y el NVIC?

Analogía: Estás cocinando (tu código principal en `main()`) y de repente tocan el timbre (evento externo). Dejas todo como está, vas a abrir la puerta (**Handler**), atendés a la persona, y luego volvéis a cocinar justo donde habías dejado.

El **NVIC** (Nested Vectored Interrupt Controller) es el controlador de tráfico de interrupciones. Decide quién tiene prioridad si tocan el timbre y suena el teléfono a la vez.

Optimizaciones del NVIC

- **Stacking:** Al entrar a la interrupción, el hardware guarda automáticamente 8 registros (R0-R3, R12, LR, PC, xPSR) en la pila. Al salir, los restaura. ¡No lo tenés que hacer a mano!
- **Tail-Chaining:** Si estás en una interrupción y llega otra esperando, en lugar de restaurar los 8 registros y volverlos a guardar, el NVIC salta directo a la segunda. Ahorra mucho tiempo (unos 6 ciclos de reloj).
- **Late Arrival:** Si llega una interrupción de prioridad baja, empieza a hacer el Stacking. Pero si justo llega una de prioridad alta, la de prioridad alta "corta fila" y se ejecuta primero sin tener que rehacer el Stacking.

Configurando EXTI (External Interrupts)

El STM32 tiene 16 líneas EXTI principales (EXTI0 a EXTI15). ¡OJO! Todos los pines 0 (PA0, PB0, PC0) van a la línea EXTI0. Hay que elegir cuál usar usando el registro `AFIO->EXTICR`.

Ejemplo: EXTI0 en PA0

```
// 1. Reloj de AFIO y GPIOA
RCC->APB2ENR |= (1<<0) | (1<<2);

// 2. PA0 como entrada Pull-Down (0x8 en CRL y 0 en ODR)
GPIOA->CRL &= ~(0xF << 0);
GPIOA->CRL |= (0x8 << 0);
GPIOA->ODR &= ~(1 << 0);

// 3. AFIO mapea PA0 a EXTI0 (bits 0-3 en 0)
AFIO->EXTICR[0] &= ~(0xF << 0);

// 4. Configurar EXTI0: Flanco de subida y habilitar
EXTI->RTSR |= (1 << 0); // Rising Trigger
EXTI->IMR  |= (1 << 0); // Habilitar mascara (Interrupt Mask)

// 5. Habilitar en el NVIC
NVIC_EnableIRQ(EXTI0_IRQn);
```

!!!CRÍTICO: Borrar la Bandera (Flag)!!!

Adentro del Handler, si no borrás la bandera, la interrupción se va a ejecutar infinitamente colgando el programa.

```
void EXTI0_IRQHandler(void) {  
    if(EXTI->PR & (1 << 0)) { // Si EXTI0 causo la interrupcion  
        // ... hacer algo (ej: togglear LED) ...  
  
        EXTI->PR = (1 << 0); // ESCRIBIR UN 1 BORRA LA BANDERA!  
    }  
}
```

Ejercicios para Practicar (Sec. 4)

Ejercicio 1: ¿Por qué si configuro PA5 y PB5 como interrupciones al mismo tiempo, el micro no me deja diferenciarlas de forma sencilla?

Ejercicio 2: ¿Cómo se limpia el bit de interrupción pendiente en el registro EXTI->PR?

Soluciones: 1) Porque ambos comparten la misma línea EXTI5. El multiplexor del AFIO solo permite conectar un pin por línea (o PA5, o PB5, no los dos a la vez a la misma línea). 2) Escribiendo un "1" en ese bit. Es anti-intuitivo, pero en los flags de estado del STM32, escribir un 1 limpia (borra) la bandera.

!!! SECCIÓN 5: Temporizadores y PWM (TIM2) !!!

¿Qué es PWM?

Pulse Width Modulation (PWM). Analogía: Si prendés y apagás la luz de tu cuarto súper rápido (más rápido de lo que ve tu ojo), podés hacer que parezca que está a media intensidad dependiendo de cuánto tiempo la dejes prendida versus apagada.

El Temporizador por dentro

Un timer como TIM2 es básicamente un contador con esteroides:

- **PSC (Prescaler):** Frena el reloj de entrada. Si dividís por 10, el timer cuenta 10 veces más lento.
- **ARR (Auto-Reload Register):** Es el límite. El timer cuenta de 0 hasta ARR y vuelve a arrancar. Esto define la Frecuencia (Período).
- **CCRx (Capture/Compare Register):** Define el Duty Cycle (Ancho del pulso). Si el contador es menor a CCR, el pin está en Alto. Si es mayor, baja.

Fórmulas clave:

$$f_{PWM} = \frac{f_{TIM}}{(PSC + 1) \times (ARR + 1)}$$

$$\text{Duty Cycle (\%)} = \frac{CCRx}{ARR + 1} \times 100$$

Pasos para PWM en TIM2 (Canal 3 y Pin PA2)

1. Prender relojes de TIM2 (en APB1) y GPIOA (en APB2).
2. PA2 configurado como Alternate Function Push-Pull (0xB).
3. Configurar TIM2 PSC y ARR.
4. Configurar TIM2->CCMR2: Poner Modo PWM 1 (0x6 en los bits OC3M).
5. Habilitar la salida en TIM2->CCER (bit CC3E).
6. Arrancar timer: TIM2->CR1 |= 1;

Ejercicios para Practicar (Sec. 5)

Ejercicio 1: Si $f_{TIM} = 72\text{MHz}$. Quiero $f_{PWM} = 1000\text{ Hz}$ (1kHz). Si fijo mi PSC = 71, ¿cuánto tiene que valer el ARR?

Ejercicio 2: Con el ARR calculado en el Ej.1, ¿qué valor pongo en el CCR3 para tener un Duty Cycle del 25 %?

Soluciones: 1) $1000 = 72,000,000 / (72 \times (ARR + 1))$. Despejando: $ARR + 1 = 72,000,000 / (72 \times 1000) = 1000$. Así que $ARR = 999$. 2) $Duty = CCR / 1000 = 0,25$. Entonces $CCR = 250$.

!!! SECCIÓN 6: Conversor ADC !!!

¿Qué es el ADC?

Analogía: Es como tomar una señal continua del mundo real (el mercurio de un termómetro subiendo suavemente, que serían voltios) y traducirla a escalones digitales (números) que el procesador pueda entender.

Características del ADC del STM32

- **Resolución:** 12 bits. Esto significa que nos va a devolver un número entre 0 y 4095 ($2^{12} - 1$).
- **Rango de Tensión:** 0V a 3.3V.
- **Cálculo de Tensión:** $V_{\text{voltaje}} = \frac{\text{Valor}_{ADC} \times 3,3V}{4095}$

Configuración del Reloj (ADCCLK)

El ADC es muy delicado y ¡NO puede funcionar a más de 14 MHz! Como nuestro bus APB2 está a 72MHz, tenemos que dividirlo usando el preescalador de ADC en el RCC (RCC->CFGR). Si dividimos por 6, nos queda un reloj de 12MHz, que está perfecto.

Proceso de Conversión (Polling)

```
// 1. Seleccionar canal (Ej: Canal 1 en SQR3)
ADC1->SQR3 = 1;

// 2. Disparar (Arrancar) la conversion (bit SWSTART)
ADC1->CR2 |= (1 << 22);

// 3. Esperar a que termine (Bandera EOC - End of Conversion)
while( !(ADC1->SR & (1 << 1)) );

// 4. Leer el resultado de Data Register
uint16_t valor = ADC1->DR;
```

(Nota: El pin de GPIO asociado al canal tiene que estar configurado como Entrada Analógica, es decir, valor 0x0 en CRL/CRH).

Ejercicios para Practicar (Sec. 6)

Ejercicio 1: Lees el ADC y te devuelve el valor 2048. ¿Cuántos voltios estás midiendo en ese pin?

Ejercicio 2: Si el bus APB2 va a 36MHz, ¿por cuánto tendrías que dividir el ADCCLK en RCC->CFGR?

Soluciones: 1) $(2048 * 3.3) / 4095 \approx 1.65V$ (justo la mitad de 3.3V, porque 2048 es la mitad de 4096). 2) Tendrías que dividir por 4 para obtener 9MHz, o dividir por 6 para obtener 6MHz (el divisor 2 da 18MHz, que es mayor al máximo de 14MHz, así que usar /4 es lo ideal).

!!! SECCIÓN 7: Comunicación Serie UART !!!

¿Qué es la UART?

Analogía: Son dos personas hablando por walkie-talkies. Acuerdan de antemano qué tan rápido van a hablar (Baud Rate). Uno tiene un cable para transmitir (TX) y el otro para recibir (RX). Los cables se cruzan: el TX de uno va al RX del otro.

Cálculo del Baud Rate

El registro BRR (Baud Rate Register) se calcula con la fórmula:

$$\text{USARTDIV} = \frac{f_{PCLK}}{16 \times \text{BaudRate}}$$

El resultado tiene una parte entera (Mantisa) que va en los bits [15:4] y una parte decimal (Fracción) que se multiplica por 16 y va en los bits [3:0].

Ejemplo para 9600 bps a 72MHz (USART1 en APB2):

1. $72,000,000 / (16 \times 9600) = 468,75$
2. Mantisa = 468 = 0x1D4. Lo desplazamos 4 bits → 0x1D40
3. Fracción = $0,75 \times 16 = 12 = 0xC$.
4. BRR = 0x1D40 — 0xC = 0x1D4C.

Transmisión, Recepción e Interrupciones

Para transmitir usamos la bandera **TXE** (Transmit Data Register Empty). Cuando está en 1, significa que el registro está libre y podemos mandar otro dato escribiendo en DR.

Para recibir usamos **RXNE** (Receive Data Register Not Empty). Cuando se pone en 1, hay un dato nuevo para leer en DR.

Para no estar bloqueando el programa esperando datos (Polling), podemos habilitar la interrupción de recepción con el bit **RXNEIE** en CR1. Cuando llega un dato, salta directo a `USART1_IRQHandler`.

Ejercicios para Practicar (Sec. 7)

Ejercicio 1: Calculá el valor del BRR para 19200 bps asumiendo $f_{PCLK} = 72\text{MHz}$.

Ejercicio 2: ¿Cómo configuro los pines PA9 (TX) y PA10 (RX)?

Soluciones: 1) $72,000,000 / (16 \times 19200) = 234,375$. Mantisa: $234 = 0xEA$. Fracción: $0,375 \times 16 = 6 = 0x6$. Por lo tanto el BRR es **0xEA6**. 2) PA9 (TX) debe ser Salida Alternate Function (**0xB**). PA10 (RX) debe ser Entrada Flotante (**0x4**).

!!! SECCIÓN 8: SPI e I2C (Sólo Teoría) !!!

SPI (Serial Peripheral Interface)

Es muy rápido. Usa 4 cables: **MOSI** (Master Out Slave In), **MISO** (Master In Slave Out), **SCK** (Reloj), y **CS/SS** (Chip Select, para elegir con quién hablás).

- Es **Full Duplex**: Pueden hablar y escuchar al mismo tiempo.
- **CPOL (Polaridad)**: Define si el reloj descansa en 0 (CPOL=0) o en 1 (CPOL=1).
- **CPHA (Fase)**: Define si se lee el dato en el primer flanco (CPHA=0) o en el segundo flanco (CPHA=1) del reloj.
- Hay 4 combinaciones posibles de CPOL y CPHA (Modos 0 al 3).

I2C (Inter-Integrated Circuit)

Es más lento pero usa solo 2 cables: **SDA** (Datos) y **SCL** (Reloj).

- Es **Half Duplex**: Hablan de a uno por vez (no a la misma vez).
- En vez de un cable para elegir al esclavo, se mandan **Direcciones** (Addressing, por lo general de 7 bits + 1 bit que indica Lectura o Escritura).
- Tiene condiciones especiales de **START** y **STOP** para iniciar y terminar la comunicación.
- El esclavo siempre responde con un **ACK** (Acuse de recibo) o **NACK** si no entendió o terminó.

Ejercicios para Practicar (Sec. 8)

Ejercicio 1: Si tenés que conectar 5 sensores a tu micro y te quedan pocos pines libres, ¿cuál elegirías, SPI o I2C?

Ejercicio 2: En SPI, si la hoja de datos de tu sensor dice "Se lee el dato en el flanco de bajada, y el reloj descansa en alto". ¿Qué CPOL y CPHA usás?

Soluciones: 1) I2C. Solo usas 2 pines en total para los 5 sensores (se conectan en paralelo al mismo bus). En SPI necesitarías 3 pines comunes + 5 pines de Chip Select (8 pines total). 2) Reloj descansa en alto $\rightarrow$ CPOL = 1. Si arranca en alto, el primer flanco es de bajada. Si se lee en el primer flanco, entonces CPHA = 0.

¡Muchísima suerte en el examen! Repasá tranquilo, vos podés.